Частное профессиональное образовательное учреждение
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА»

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 5

По дисциплине: «Системное программирование»

Тема: «Кодировки текста: запись и чтение файла в UTF-8»

Выполнил: студент
Группы: ИСиП-23-1
ФИО: Зыков Егор Тимурович

Проверил: преподаватель
Кузнецов А.Н.
Оценка: _____
Дата: «12» ноября 2026 г.

Сургут, 2026

1. Тема, цель и основные понятия

Тема: Кодировки текста: запись и чтение файла в UTF-8.

Цель: Научиться записывать и читать кириллический текст с явной UTF-8, сравнивать исходную и прочитанную строки, получать bytes и объяснять различия корректного и ошибочного декодирования.

str — последовательность символов Unicode, bytes — последовательность байтов. Кодирование преобразует символы в байты; декодирование выполняет обратное преобразование согласно выбранной кодировке. UTF-8 использует переменную длину: символы ASCII занимают один байт, большинство русских букв — два. Поэтому длина строки и объём её UTF-8 представления могут отличаться.

Средства реализации: Python 3 и стандартная библиотека pathlib.

2. Выполнение учебных заданий

2.1. Задание 1. Подготовить UTF-8 файл

Внутри Path.cwd() создаётся encoding_lab/texts/student.txt минимум с двумя строками кириллицы. encoding='utf-8' явно задаёт правило записи символов в байты.

Фактический результат: Создан encoding_lab/texts/student.txt с двумя строками кириллицы. Журнал сообщает «Файл записан в UTF-8» и «Существует: True»; содержимое файла проверено чтением UTF-8.

2.2. Задание 2. Записать собственный текст

В encoding_lab/texts/profile.txt записываются минимум три строки: имя, группа и русская фраза. Проверяются существование файла и читаемость кириллицы.

Фактический результат: Создан profile.txt: «Имя: Зыков Егор Тимурович.», «Группа: ИСиП-23-1.», «Изучаю хранение русского текста в файле.». Файл существует, его размер — 152 байта; строки прочитаны корректно.

2.3. Задание 3. Прочитать profile.txt

Файл открывается в режиме r с UTF-8. Прочитанная строка сравнивается с исходной; одинаковая кодировка обеспечивает обратное преобразование байтов в нужные символы.

Фактический результат: Консоль вывела три исходные строки profile.txt и «Прочитанное совпадает с исходным: True». Совпадение подтверждено при обоих запусках exercises.

2.4. Задание 4. Получить байты строки

Строка «Кодировка UTF-8 работает с кириллицей» преобразуется encode('utf-8'), затем восстанавливается decode('utf-8'). Сравнение проверяет сохранность текста при обратном преобразовании.

Фактический результат: Для строки «Кодировка UTF-8 работает с кириллицей» выведен объект bytes и восстановленная строка. Результат «Восстановленная строка совпадает: True» подтверждает обратимость UTF-8.

2.5. Задание 5. Сравнить две кодировки

Для строки «Файл и кодировка» одни и те же UTF-8 байты декодируются через utf-8 и latin-1. latin-1 интерпретирует каждый байт по другому правилу и искажает кириллицу; сами исходные байты не меняются.

Фактический результат: UTF-8 декодирование дало «Файл и кодировка», latin-1 — «Ð¤Ð°Ð¹Ð» Ð¸ ÐºÐ¾Ð´Ð¸Ñ\x80Ð¾Ð²ÐºÐ°» (непечатаемый символ показан как \x80). Сравнения с оригиналом: True и False соответственно; байты для двух попыток были одними и теми же.

2.6. Задание 6. Прочитать profile.txt как bytes

Path.read_bytes() возвращает байты без декодирования. Выводятся первые 20 байтов, затем выполняется decode('utf-8') и сравнение с обычным текстовым чтением. Переводы строк учитываются отдельно: текстовый режим может нормализовать CRLF.

Фактический результат: profile.txt прочитан как bytes: журнал вывел первые 20 байтов и размер 152. Декодирование UTF-8 восстановило те же три строки, которые показало текстовое чтение; обе проверки совпадения с оригиналом = True.

3. Итоговая самостоятельная программа

Название: «Проверка кодировки текстового файла».

Создан encoding_check/note.txt с собственным текстом из трёх строк. Фактически записанный и прочитанный текст:
Автор заметки: Зыков Егор Тимурович.
Группа ИСиП-23-1 изучает системное программирование.
Сегодня проверяется сохранение русского текста в UTF-8.

Сравнение прочитанного с исходным и проверка UTF-8 декодирования дали True; latin-1 совпадение дало False. Файл содержит 261 байт. Первые 20 байтов: b'\xd0\x90\xd0\xb2\xd1\x82\xd0\xbe\xd1\x80 \xd0\xb7\xd0\xb0\xd0\xbc\xd0\xb5\xd1'. Повторный запуск дал тот же текст и те же результаты; оба полных журнала сохранены рядом с программой.

3.1. Полный исходный код main.py

```
"""Практическая №5. Проверка кодировки. Зыков Е. Т., ИСиП-23-1."""

import argparse
from pathlib import Path

NOTE_TEXT = (
    "Автор заметки: Зыков Егор Тимурович.\n"
    "Группа ИСиП-23-1 изучает системное программирование.\n"
    "Сегодня проверяется сохранение русского текста в UTF-8.\n"
)
EXTRA_TEXT = "Байты хранят текст, а кодировка объясняет их значение.\n"


def checkpoint(message, pause=False):
    print("\n=== " + message + " ===", flush=True)
    if pause:
        input("Enter — продолжить: ")


def write_utf8(path, text):
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8", newline="\n") as file:
        file.write(text)
    print("Файл записан в UTF-8:", path)
    print("Существует:", path.exists())


def read_utf8(path):
    with open(path, "r", encoding="utf-8", newline="\n") as file:
        return file.read()


def inspect_encoding(path, original):
    loaded = read_utf8(path)
    data = path.read_bytes()
    decoded_utf8 = data.decode("utf-8")
    decoded_latin1 = data.decode("latin-1")
    print("Прочитанный текст:\n" + loaded)
    print("Прочитанное совпадает с исходным:", loaded == original)
    print("Первые 20 байтов:", data[:20])
    print("Количество байтов:", len(data))
    print("UTF-8:\n" + decoded_utf8)
    print("latin-1:\n" + decoded_latin1)
    print("UTF-8 совпадает с исходным:", decoded_utf8 == original)
    print("latin-1 совпадает с исходным:", decoded_latin1 == original)
    print("UTF-8 восстанавливает кириллицу; latin-1 трактует каждый байт как отдельный символ.")
    return {
        "loaded": loaded, "data": data, "utf8": decoded_utf8,
        "latin1": decoded_latin1, "matches": loaded == original,
    }


def main(pause=False):
    print("Рабочая папка:", Path.cwd())
    base = Path.cwd() / "encoding_check"
    checkpoint("Перед записью UTF-8", pause)
    write_utf8(base / "note.txt", NOTE_TEXT)
    checkpoint("note.txt записан: можно открыть в редакторе", pause)
```

```
    inspect_encoding(base / "note.txt", NOTE_TEXT)
    checkpoint("Чтение и декодирование проверены", pause)
    extra = base / "extra.txt"
    write_utf8(extra, EXTRA_TEXT)
    data = extra.read_bytes()
    print("Дополнительный текст:\n" + EXTRA_TEXT)
    print("Символов:", len(EXTRA_TEXT))
    print("Байтов:", len(data))
    print("Кириллический символ в UTF-8 занимает больше одного байта.")
    checkpoint("Сравнение символов и байтов завершено", pause)
    return base


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Проверка кодировки текстового файла")
    parser.add_argument("--pause", action="store_true", help="ожидать Enter между фазами")
    main(pause=parser.parse_args().pause)
```

4. Самостоятельное изменение

Создан encoding_check/extra.txt с текстом «Байты хранят текст, а кодировка объясняет их значение.» и завершающим переводом строки. Фактическая длина — 55 символов, включая перевод строки, и 100 UTF-8 байтов. Кириллические буквы занимают больше одного байта, а пробелы, запятая, точка и перевод строки — по одному; поэтому байтов больше, чем символов.

5. Подтверждение выполнения

Фактические результаты приведены в разделе выполнения заданий и таблице наблюдений. Источниками подтверждения служат реальные журналы запуска, сохранённые отдельно рядом с программами:

outputs\exercises_run1.txt

outputs\exercises_run2.txt

outputs\main_run1.txt

outputs\main_run2.txt

outputs\tests.txt

Скриншоты выполнения в этот отчёт не включены: по согласованному ограничению подтверждение представлено фактическими текстовыми результатами и отдельными журналами outputs. Это не является полным соответствием требованию исходного задания о скриншотах; изображения консоли и Проводника не подменялись искусственно созданными иллюстрациями.

Выдержка: outputs/main_run1.txt

```
Прочитанное совпадает с исходным: True
Первые 20 байтов: b'\xd0\x90\xd0\xb2\xd1\x82\xd0\xbe\xd1\x80 \xd0\xb7\xd0\xb0\xd0\xbc\xd0\xb5\xd1'
Количество байтов: 261
```

Выдержка: outputs/main_run1.txt

UTF-8 совпадает с исходным: True
latin-1 совпадает с исходным: False

Выдержка: outputs/main_run1.txt

Символов: 55
Байтов: 100

Выдержка: outputs/tests.txt

Ran 5 tests in 0.032s

OK

6. Таблица реальных наблюдений

Наблюдение	Фактическое значение	Пояснение
Путь к note.txt	G:\Мой диск\Колледж\Сиспрог\College\Практики по системному программированию\Практическая_5\encoding_check\note.txt	Файл существует внутри папки практической.
Исходный текст	Автор заметки: Зыков Егор Тимурович. Группа ИСиП-23-1 изучает системное программирование. Сегодня проверяется сохранение русского текста в UTF-8.	Три строки NOTE_TEXT; завершающий перевод строки сохранён.
Прочитанный текст	Автор заметки: Зыков Егор Тимурович. Группа ИСиП-23-1 изучает системное программирование. Сегодня проверяется сохранение русского текста в UTF-8.	Чтение с UTF-8 восстановило исходные строки.
Совпадение строк	Чтение = True; decode UTF-8 = True	Оба запуска main дали эти значения.
Количество байтов	261	len(read_bytes()) = 261.
Результат latin-1	Ð\x90Ð²Ñ\x82Ð¾Ñ\x80 Ð·Ð°Ð¼ÐµÑ\x82Ðº… (фрагмент; полный вывод: outputs/main_run1.txt)	Ошибочная строка сохранена; только C1 управляющие показаны как \xNN. Совпадение False.

Наблюдение	Фактическое значение	Пояснение
Самостоятельное изменение	extra.txt: 55 символов; 100 байтов	Текст с завершающим LF; кириллица кодируется несколькими байтами.

7. Контрольные вопросы и ответы

1. Что такое кодировка текста?

Правило преобразования символов в байтовую последовательность и её обратной интерпретации.

2. Для чего в open() указывается encoding='utf-8'?

Чтобы явно выбрать UTF-8 для кодирования при записи или декодирования при чтении и не зависеть от кодировки по умолчанию.

3. Что делает метод encode()?

Преобразует строку str в объект bytes по указанной кодировке, например text.encode('utf-8').

4. Что делает метод decode()?

Преобразует bytes в строку str по указанной кодировке. Неподходящая кодировка может дать искажённый текст или ошибку.

5. Почему кодировка при записи и чтении должна совпадать?

Байты должны интерпретироваться по тем же правилам, которыми они получены. Иначе исходные символы могут не восстановиться.

6. Чем str отличается от bytes в этой работе?

str содержит символы, а bytes — их закодированное байтовое представление. len(str) считает кодовые точки, len(bytes) — байты.

7. Что возвращает Path.read_bytes()?

Полное содержимое файла как bytes без текстового декодирования и нормализации переводов строк.

8. Почему декодирование UTF-8-байтов через latin-1 даёт другой текст?

latin-1 независимо отображает каждый байт в символ, а UTF-8 читает многобайтовые последовательности. Разные правила дают разные символы при неизменных байтах.

8. Вывод

Текст с кириллицей записан и прочитан с явным указанием UTF-8. Полученная строка совпала с исходной, а файл note.txt занял 261 байт. Декодирование тех же байтов через latin-1 исказило текст и дало отрицательное сравнение. Дополнительный extra.txt показал различие между 55 символами и 100 байтами, включая завершающий перевод строки.